Práctica 4. Programa para realizar paso a paso multiplicaciones de 3 cifras

Manuela Planelles Lucas

1º DAW – 2024/25 – Programación

1. Objetivo de la practica 2
2. Programa en Java 3
1.1. Pasos seguidos 4
1.2. Explicación del código 4
3. Plan de pruebas 6
3.1. Prueba con números negativos 6
3.2. Prueba introduciendo ceros 7
3.3. Prueba con números 1, 2 u 4 cifras 7
3.4. Prueba correcta – caso feliz 8

GitHub

1. **Objetivo de la practica**

   El objetivo de esta práctica es implementar un programa en Java que muestre el proceso paso a paso de una multiplicación de dos números de 3 cifras, tal como se hace a mano. El programa pide los dos números, valida que sean de exactamente 3 cifras, calcula el producto y muestra los resultados parciales desplazados. Se trabaja con métodos, conversión entre tipos de datos con Integer.toString(), Integer.parseInt() y substring(), y control de errores con try-catch y bucle do-while.

2. Programa en Java

multiplicacion3Cifras.java

```java
import java.util.InputMismatchException;
import java.util.Scanner;

public class multiplicacion3Cifras {  MANUELA\manue *
    static Scanner teclado = new Scanner(System.in);  2 usages
    public static void main(String[] args) {  MANUELA\manue
        menu();
    }
    public static void menu() {  MANUELA\manue
        int num1 = leerNumero( mensaje "Introduce el multiplicando (3 cifras): ");
        int num2 = leerNumero( mensaje "Introduce el multiplicador (3 cifras): ");

        System.out.println("El producto de la multiplicación es: " + multiplicacion(num1, num2));
        proceso(num1, num2);
        System.out.println("_____");
        System.out.println(multiplicacion(num1, num2));
    }
    public static int leerNumero(String mensaje) {  2 usages  MANUELA\manue *
        int num = 0;
        boolean valido = false;
        do {
            try {
                System.out.println(mensaje);
                num = teclado.nextInt();
                if (num < 0) {
                    System.out.println("No se permiten números negativos.");
                } else if (num < 100 || num > 999) {
                    System.out.println("Debe tener exactamente 3 cifras (entre 100 y 999).");
                } else {
                    valido = true;
                }
            } catch (InputMismatchException e) {
                System.out.println("Error: debe introducir un número entero.");
                teclado.next();
            }
        } while (!valido);
        return num;
    }
    public static int multiplicacion(int num1, int num2) {  2 usages  MANUELA\manue
        return num1 * num2;
    }
    public static void proceso(int num1, int num2) {  1 usage  MANUELA\manue
        String numStr = Integer.toString(num2);

        int num_3 = Integer.parseInt(numStr.substring(2, 3)); // unidades
        int num_2 = Integer.parseInt(numStr.substring(1, 2)); // decenas
        int num_1 = Integer.parseInt(numStr.substring(0, 1)); // centenas

        System.out.println("El proceso es: ");
        System.out.println("  " + num1);
        System.out.println("x " + num2);
        System.out.println("_____");
        System.out.println("  " + num1 * num_3);
        System.out.println(num1 * num_2 + "x");
        System.out.println(num1 * num_1 + "xx");
    }
}
```

1.1. Pasos seguidos

Al principio lo que más me costó fue entender el paso a paso de lo que pedía la práctica. Una vez que lo entendí, decidí usar los métodos para marcar ese paso a paso y verlo con más claridad, en lugar de escribirlo todo junto en el main.

Primero creé el método menu(), que es el que controla el flujo general y llama al resto. Desde ahí llamo a leerNumero(), que se encarga de pedir y validar cada número. Luego tengo multiplicacion(), que es la acción más sencilla, solo calcula el resultado. Y por último proceso(), que fue el que más me costó entender porque tenía que imprimir los resultados parciales desplazados exactamente como pide el enunciado.

1.2. Explicación del código

- Método menu()

Es el método principal, el que se llama desde el main. Su función es coordinar el resto: llama a leerNumero() dos veces para obtener los dos números válidos, luego llama a multiplicacion() para mostrar el producto y a proceso() para mostrar el paso a paso. No recibe nada y no devuelve nada (void).

```
11      public static void menu() {   MANUELA\manue
12          int num1 = leerNumero( mensaje "Introduce el multiplicando (3 cifras): ");
13          int num2 = leerNumero( mensaje "Introduce el multiplicador (3 cifras): ");
14
15          System.out.println("El producto de la multiplicación es: " + multiplicacion(num1, num2));
16          proceso(num1, num2);
17          System.out.println("_____");
18          System.out.println(multiplicacion(num1, num2));
19      }
```

- Método leerNumero(String mensaje)

Recibe el texto que quiero mostrar por pantalla y devuelve (return) un int con el número válido. He usado un bucle do-while en lugar de un while normal porque necesito que el cuerpo se ejecute al menos una vez: primero pide el número y luego comprueba si es válido. Si usara un while normal tendría que pedir el número antes del bucle y otra vez dentro, lo cual sería repetir código. El try-catch captura la excepción InputMismatchException, que es el error que lanza Java cuando el usuario introduce letras en lugar de números. Sin ese control el programa petaría directamente.

```java
public static int leerNumero(String mensaje) {  2 usages  MANUELA\manue *
    int num = 0;
    boolean valido = false;
    do {
        try {
            System.out.println(mensaje);
            num = teclado.nextInt();
            if (num < 0) {
                System.out.println("No se permiten números negativos.");
            } else if (num < 100 || num > 999) {
                System.out.println("Debe tener exactamente 3 cifras (entre 100 y 999).");
            } else {
                valido = true;
            }
        } catch (InputMismatchException e) {
            System.out.println("Error: debe introducir un número entero.");
            teclado.next();
        }
    } while (!valido);
    return num;
}
```

- Método multiplicacion(int num1, int num2)

Recibe los dos números y devuelve (return) su producto. Es el método más sencillo. Al devolver un int puedo usarlo directamente dentro de un println sin necesidad de guardar el resultado en una variable extra.

```java
public static int multiplicacion(int num1, int num2) {  2 usages  MANUELA\manue
    return num1 * num2;
}
```

- Método proceso(int num1, int num2)

Recibe los dos números y no devuelve nada (void) porque solo imprime por pantalla. Este fue el que más me costó. Para sacar cada cifra del multiplicador por separado primero lo convierto a String con Integer.toString(), luego uso substring() para coger cada posición: substring(2,3) para las unidades, substring(1,2) para las decenas y substring(0,1) para las centenas. Después convierto cada cifra de vuelta a entero con Integer.parseInt() para poder multiplicar. Antes de corregir el código, si el multiplicador tenía menos de 3 cifras el substring(2,3) petaba con un StringIndexOutOfBoundsException porque el String no tenía esa posición. Por eso la validación en leerNumero() es necesaria antes de llegar aquí. Las x al final de los resultados parciales representan los ceros desplazados de la multiplicación tradicional, tal como pide el enunciado.

```java
    public static void proceso(int num1, int num2) {  1 usage  MANUELA\manue
        String numStr = Integer.toString(num2);

        int num_3 = Integer.parseInt(numStr.substring(2, 3)); // unidades
        int num_2 = Integer.parseInt(numStr.substring(1, 2)); // decenas
        int num_1 = Integer.parseInt(numStr.substring(0, 1)); // centenas

        System.out.println("El proceso es: ");
        System.out.println("  " + num1);
        System.out.println("x " + num2);
        System.out.println("_____");
        System.out.println("  " + num1 * num_3);
        System.out.println(num1 * num_2 + "x");
        System.out.println(num1 * num_1 + "xx");
    }
}
```

3. Plan de pruebas

3.1. Prueba con números negativos

Introduce: -5 como multiplicando. El bucle detecta num < 0, muestra el aviso y vuelve a pedir el número.

```
Introduce el multiplicando (3 cifras):
-5
No se permiten números negativos.
Introduce el multiplicando (3 cifras):
```

3.2. Prueba introduciendo ceros

Introduce: 0 como multiplicando. Entra en la condición num < 100, muestra el error y vuelve a pedir.

```
Introduce el multiplicando (3 cifras):
0
Debe tener exactamente 3 cifras (entre 100 y 999).
Introduce el multiplicando (3 cifras):
```

3.3. Prueba con números 1, 2 u 4 cifras

Introduce: 12 primero, luego 1234. En ambos casos el bucle muestra el error y vuelve a pedir. Antes de corregir el código, introducir 12 causaba un StringIndexOutOfBoundsException porque proceso() intentaba hacer substring(2,3) sobre un String de solo 2 caracteres.

```
Introduce el multiplicando (3 cifras):
12
Debe tener exactamente 3 cifras (entre 100 y 999).
Introduce el multiplicando (3 cifras):
1234
Debe tener exactamente 3 cifras (entre 100 y 999).
Introduce el multiplicando (3 cifras):
```

3.4. Prueba correcta – caso feliz

Introduce: 367 y 251. El programa muestra el producto, el proceso con los resultados parciales desplazados y el resultado final.

```
Introduce el multiplicando (3 cifras):
367
Introduce el multiplicador (3 cifras):
251
El producto de la multiplicación es: 92117
El proceso es:
  367
x 251
-----
  367
1835x
734xx
-----
92117

Process finished with exit code 0
```